Five Real Projects. Two Companies.

Two Industry Partners × DigiPen (Singapore)

An early look at the industry projects available in Software Engineering Project 5/6 — and how to put your name forward

Read this first

Two companies have brought us real problems they are trying to solve. Between them there are five projects on offer, and each one is looking for a student team.

Every one of the five is a 3D application that runs in a browser. Some of them may not sound like it at first — one is about electric vans, another about electricity bills — but all five put a 3D scene at the centre of the screen and ask you to make something complicated understandable by letting people see it.

This document is an early look. It tells you what each project is about, what you would build, and what you would learn. It does not contain the full specifications — those come later, and Section 8 explains why.

You will also notice that the two companies are not named here. Their identities, like the full project details, are shared once the formal agreements are signed.

You do not need to understand every project to apply. You need to find one that makes you curious.

Why apply to an industry project

These are not exercises invented for a classroom. They are problems real companies are trying to solve, with real users waiting at the end of them. You will have an industry mentor, a genuine deadline, and something in your portfolio that a future employer will actually ask you about.

1. The two partners

Partner A — energy, electric vehicles and digital twins

Partner A works on the electric vehicle transition. Their platform helps businesses figure out how to move their vehicle fleets from petrol and diesel to electric — which vehicles to change, when, where they will charge, what it costs, and what it does to emissions.

The reason their work is 3D is that all of these decisions happen in real places. A depot is a physical yard with parking bays, chargers and a power connection. A route runs across real terrain with real hills. A charging network is spread across a real city. Showing a fleet manager a spreadsheet tells them a number; showing them their own depot, filling with electric vans year by year, tells them what the number means.

This is a large and fast-growing industry. Every delivery company, bus operator and logistics business in the region will face these decisions within a few years, and most of them have no idea how to make them.

Partner B — 3D and spatial software

Partner B builds software for 3D and spatial content. Their work sits where games technology meets everything else — museums, education, heritage, training. They are interested in taking the tools game developers take for granted and putting them in the hands of people who have never written a line of code.

If you have spent years learning 3D, this is a chance to point those skills somewhere unexpected.

2. The five projects at a glance

Project	Partner	In one line	The 3D in it
1. Fleet Transition Planner	A	Decide which vehicles to electrify, and when	A digital twin of the depot yard, changing year by year
2. Route & Charging Optimiser	A	Choose the best charger along a journey	A 3D map with terrain, routes and charger sites
3. Energy & Sustainability Simulator	A	See what happens to power, cost and emissions across sites	A city-scale 3D network with demand rising and falling over time
4. 3D Digital Heritage Platform	B	Document and compare 3D scans of real museum objects	Real scanned models, with notes anchored to points on the surface
5. Spatial Experience Builder	B	Let anyone build a 3D experience without coding	A full 3D editor with drag handles, scenes and a published runtime

Why all five are 3D projects

It is worth being clear about this, because two of the five are about energy and vehicles and might read as spreadsheet work.

They are not. In all five projects, the 3D view is not decoration added at the end — it is how the user understands the problem. A number in a table tells a fleet manager that their evening peak demand is 340 kW. A 3D depot showing twenty vans plugged in at once, with the site's power ceiling visible above them, tells them why it matters and what to do about it.

Practically, that means every team will work with real-time rendering in the browser, a scene that responds to changing data, camera control, and the performance problems that come with drawing a lot on screen at once. If those are the skills you have been building, all five of these projects want them.

> **A note on digital twins**
>
> Three of these projects are, in effect, digital twins — a live 3D model of something real, fed with real or simulated data, used to answer questions about the real thing. This is one of the strongest growth areas in the industry right now, and it sits almost exactly on top of what a games education teaches you.

Project 1 — Fleet Transition Planner

Partner A · Which vehicles should we electrify, when, and what will it cost?

A company owns one hundred vans. Some are new, some are ten years old. Some drive 200 km a day, some barely leave the yard. Management has decided to go electric. Nobody knows where to start.

Here is what makes it a real problem rather than a simple one. An electric van costs more to buy and less to run. Whether that trade works out depends almost entirely on how far the van is driven. A van doing 40,000 km a year pays for itself in about three years. An identical van doing 6,000 km a year may never pay for itself at all. Same vehicle, same prices, opposite answer.

So there is a correct answer, and it is different for every vehicle. Add the question of *when* — this year, next year, in three years — and with a hundred vehicles you have more possible plans than anyone could ever work through on paper.

Where the 3D comes in

Electrifying a fleet is not only a financial decision. It is a physical one. Electric vans need chargers, chargers need space in the yard, and the yard has a fixed power connection that cannot be exceeded. A plan that works in a spreadsheet can be impossible in the actual depot.

So the centre of this application is a **3D model of the depot** — a digital twin of a real place. You see the parking bays, the vehicles in them, the chargers and where they are installed. As the user moves through the transition years, the yard changes in front of them: diesel vans become electric, chargers appear, and the power draw at the site climbs toward its ceiling. Place one charger too many and the site turns red.

That is the moment the tool earns its keep. It turns an abstract plan into something a manager can look at and immediately understand.

What you would build

- A 3D depot view showing the fleet, the parking layout and the charging infrastructure
- A vehicle list where the user selects vehicles or whole categories and assigns each a transition year
- A calculation engine that recomputes fleet cost, payback and emissions instantly as choices change
- Editable assumptions — fuel price, electricity price, vehicle cost, charger cost — so the user can test different futures
- A timeline the user scrubs through, with the depot updating to match the selected year
- Side-by-side comparison of two transition plans, so "A now versus B next year" has a visible answer

What you would learn

- Building a real-time calculation engine that stays responsive under constant change
- Driving a 3D scene from live data rather than from a fixed script
- Designing charts and comparisons that a non-technical professional actually trusts
- Making a recommendation explain itself, instead of asking the user to take it on faith

Project 2 — Route & Charging Optimiser

Partner A · This vehicle needs to charge. Which charger should it use?

A van is halfway through its deliveries and the battery is getting low. Three chargers are within reach.

The first is 1 km away, fast, and expensive. The second is 5 km off the route, the cheapest by some distance, and slow. The third is close and mid-priced, but there is already a fifteen-minute queue.

There is no correct answer here. There is only a correct answer once you know what matters — and what matters keeps changing. If there is a delivery deadline, the driver's time costs far more than the few dollars saved at the cheap charger. If it is the last run of the day, take the cheap one and wait.

This is what makes the project interesting: it is a genuine multi-objective problem, the kind where optimising one thing makes another worse, and the software has to help a human make a judgement rather than pretend there is a single right answer.

Where the 3D comes in

Everything in this problem is geographic, and flat maps hide half of it.

The application is built on a **3D map** — real terrain, real buildings, the route drawn across the landscape and the charging sites standing on it. The camera can follow the journey. A five-kilometre detour stops being a number and becomes a visible loop away from where the driver needs to be.

Terrain matters for a second reason too. Hills change how much energy a vehicle uses. A route over a ridge drains the battery faster than the same distance on the flat, which means elevation is not just something to look at — it is an input to the calculation. Working with real geospatial data, at real coordinates, is a valuable and surprisingly rare skill.

What you would build

- A 3D map view with terrain, a planned route, and charging sites placed along or near it
- A charger comparison panel showing power, price, expected queue, detour distance and charging time
- A scoring system that weighs those factors together and produces a ranked recommendation
- Priority controls — cheapest, fastest, shortest detour, most reliable, best overall — that visibly change the ranking
- A plain-language explanation of why the winning charger won, and what choosing it cost
- Optionally, several vehicles at once — because the best charger for one van stops being best when another van is already sitting on it

What you would learn

- Geospatial 3D: terrain, coordinate systems, and streaming map data efficiently
- Multi-objective scoring — how to combine factors that are measured in different units
- Building an intelligent recommendation that a person can inspect and disagree with
- Camera work and animation in service of explanation rather than spectacle

Project 3 — Energy & Sustainability Simulator

Partner A · What happens when the whole fleet plugs in at once?

Here is something most people have never thought about. A business does not only pay for the electricity it uses. It also pays for its single highest moment of demand across the whole billing period — and every site has a hard physical limit it is not allowed to exceed.

Twenty electric vans all plugging in at six in the evening creates a spike. That spike can cost more than the energy itself. Worse, it can exceed what the building is physically permitted to draw, at which point the answer is not "pay more" but "you cannot", and the company is looking at a grid upgrade that takes months and costs a fortune.

Spread exactly the same charging across the night and the problem disappears entirely. Nothing about the vehicles changed. Only the timing did.

That gap — between a plan that fails and an identical plan that works — is what this project makes visible.

Where the 3D comes in

A fleet does not charge in one place. It charges at its own depots, at public charging sites, and at partner locations spread across a city. The interesting effects are the ones that appear across the whole network: move charging away from one depot and it lands somewhere else.

So this is a **city-scale digital twin**. Sites sit on a 3D map at their real locations. Demand at each site is shown spatially — rising and falling, colour-shifting as it approaches capacity. A time slider runs through the day and you watch the evening peak sweep across the network, site by site. Where a site hits its ceiling, you see it happen rather than reading it in a log.

Then the user starts experimenting. Shift charging to overnight. Add a battery to one depot. Move twenty percent of the load to a public site. The network responds, and the consequences are visible immediately.

What you would build

- A 3D city view with depots and public charging sites at their real positions
- A simulation of charging demand across a full day, site by site
- Visual encoding of load, cost and capacity — so an overloaded site is obvious at a glance
- A time control that lets the user scrub through the day and watch the peak move
- Scenario controls: how much of the fleet is electric, when it charges, what each site charges per unit
- Optional battery storage and solar, so the user can test whether they solve the peak problem
- Emissions output alongside cost, comparing the electric fleet against the diesel baseline

What you would learn

- Simulating a system over time and keeping it fast enough to feel interactive
- Working with time-series data and making it legible in three dimensions
- Digital twin architecture — connecting a live data model to a spatial view
- A genuinely interesting corner of how energy systems and cities actually work

Project 4 — 3D Digital Heritage Platform

Partner B · How do you keep a permanent record of a physical object that is slowly falling apart?

A museum has a three-hundred-year-old wooden statue. It is cracking, slowly, the way wood does. Someone scans it in 3D, accurate to less than a millimetre. Two years later they scan it again, so they can see exactly what changed.

Then this happens. The scan becomes a file on a shared drive, named something like `statue_final_v3_REAL_final.glb`. It sits in a folder next to five similar files. Nobody can say which physical object it belongs to, who scanned it, or which file is the good one. The conservator's notes about the crack on the left shoulder live in a Word document somewhere else entirely. The photograph of that crack is a third file called IMG_4471.jpg.

Two years of work, an irreplaceable object, real money spent — and a record nobody can use.

The people who look after these objects are called conservators. Their whole job is documenting what is happening to something over decades. Right now the software fails them completely.

Where the 3D comes in

This project is 3D all the way down, and the hard part is more interesting than it sounds.

The centre of the application is the scanned model itself. A conservator opens it, rotates it, zooms in on the surface — and then clicks directly on the crack to leave a dated note exactly there. That note must stay on that exact spot forever: after the camera moves, after the window resizes, after someone opens it on a phone two years later.

Getting that right means turning a flat mouse click into a real point on a 3D surface, storing it in the model's own coordinate system rather than on the screen, and hiding markers that are on the far side of the object. Then there is comparison: two scans of the same statue, opened side by side, cameras locked together so that moving one moves the other. That is when change becomes visible instead of merely described.

What you would build

- A browser 3D viewer that loads real scanned models and stays smooth on an ordinary laptop
- Records that represent the physical object, with scans, photographs and notes attached to them over time
- Click-to-place annotations anchored to exact points on the model surface, with categories and severity
- A conservation timeline showing what was observed and treated, and when
- Side-by-side comparison of two scans with synchronised cameras
- Search, permissions, and the ability to publish a record for the public to view but not edit

What you would learn

- Real 3D on the web — loading, rendering and performing well with large models
- Raycasting and coordinate maths: turning a click into a point that survives everything
- Occlusion and screen-space layout, so fifteen markers stay readable
- Designing data that has to remain meaningful for decades, not just for the demo

Project 5 — Spatial Experience Builder

Partner B · How does someone who cannot code build a 3D experience?

A history teacher wants her class to build a virtual exhibition. The students have done the research — they know the artefacts, the buildings, the story. They want a space a visitor can walk through, where clicking a crate reveals what was inside it and standing near the river starts a narration.

Her options today are all bad. Slides, which lose every sense of place. Hire a developer, which costs thousands, takes months, and teaches her students nothing. Or learn a game engine, which takes longer than the term she has.

There is nothing in between. Someone who knows the content has no way to build a spatial experience without first becoming a programmer.

Here is the part worth understanding before you apply. Anyone can build a 3D scene if they are allowed to write code — loading a model and adding a click handler is an afternoon's work. The difficulty in this project is that **the author is not allowed to write code**. Every behaviour they might want has to be expressible through an interface you design. That constraint is what makes it hard, and it is the same problem Scratch, Webflow and Unreal's Blueprints each had to solve in their own way.

Where the 3D comes in

You are building a 3D editor — the kind of software you have spent years using without seeing inside.

The author opens a 3D space, drags objects in from a library, and moves them with on-screen handles. Those handles look trivial and are not: picking the right one, dragging along the correct axis from any camera angle, keeping a constant size on screen, handling the awkward case where the camera looks straight down the axis being dragged. Then everything the author builds has to be saved into a file and reloaded perfectly — and the published version, which visitors open from a link, is a separate program reading that same file. If those two ever disagree, the tool is broken in the worst possible way, because it fails silently.

What you would build

- A browser-based 3D editor: scene tree, asset library, 3D canvas and property panel
- Move, rotate and scale handles that behave correctly from any viewpoint
- Content the author can attach — information panels, images, audio, video, hotspots
- A simple rule builder: when the visitor clicks this, do that; when they enter this area, play that
- Multiple scenes with navigation between them, plus undo, redo and autosave
- Publishing to a link that opens on any phone, with no editing capability in the published version

What you would learn

- How editors actually work, from the inside — the most transferable tooling skill there is
- Transform gizmo mathematics, which is far harder and more satisfying than it looks
- Saving and reloading a scene perfectly, and versioning that format as it changes
- Event systems, and designing a small vocabulary that non-technical people can master

3. Who should apply

All five projects need a full team, and every discipline has real work to do.

If you are…	There is work here for you
UXGD	These products are used by professionals doing real jobs — conservators, fleet managers, teachers, school students. Understanding those users and designing for them is the difference between a tool people use and one they abandon. You would also lead the team as project manager
IMGD	3D views, interactive maps, dashboards, charts, editors and visual feedback. All five projects live or die on whether the interface makes a complicated thing feel simple
RTIS	Every project has a genuine engine at its centre — a calculation engine, a scoring system, a simulation, a 3D anchoring system, an editor runtime. This is the technically hardest work and the most portfolio-worthy
BFA	Icons, visual language, 3D asset libraries, environment dressing, and the polish that makes a demo memorable rather than merely functional

What we are really looking for is not people who already know how to build these things. Nobody does yet. We are looking for people willing to learn something new — a framework they have not used, a domain they know nothing about, a problem with no obvious answer.

> *This is the lowest-risk chance you will ever get to take. Here, learning something new costs you a few uncomfortable weeks. In industry, it costs you a role you were not ready for.*

4. Important notice — intellectual property

Please read this section carefully before applying. It matters, and we would rather you knew now than later.

These are the company's projects, and the work you produce contributes to the company's intellectual property.

Unlike a project you invent yourself, these problems belong to our industry partners. The software your team builds — the code, the designs, the models and the documentation — becomes intellectual property under an agreement between DigiPen (Singapore) and the company.

That agreement is a General Framework Agreement, signed by DigiPen and the partner company before the project begins. It sets out ownership, licensing, confidentiality and your rights.

What this means for you in practice:

- **You will be credited.** DigiPen (Singapore) and the student team must be credited in the project. The company cannot present the work as having been built by its own in-house team.
- **You can showcase it.** You are permitted to put the project in your portfolio — your website, GitHub, LinkedIn — and to talk about it in interviews. This is protected in the agreement, not left to goodwill.
- **You may work with confidential material.** Some information shared with your team may not be public. You may be asked to sign a non-disclosure agreement, and you will need to handle that material properly.
- **You cannot take the product and launch it yourself.** The problem belongs to the company. What you take away is the experience, the skills, and the right to show what you built.

If you are not comfortable with these terms, that is a perfectly reasonable position — and there are other project routes in the module that do not involve an industry partner. Please talk to us early rather than joining and discovering it later.

5. Why the full details are not in this document

You may be wondering why this document describes the problems but not the specifications.

The reason is straightforward. The detailed briefs contain the companies' product plans, their commercial thinking, and in some cases sample data and material they have not made public. That information — and the companies' names — is shared once the formal agreements are in place, not before.

Stage	What you get
Now — expression of interest	This document. Enough to know which projects interest you and whether you want in
After agreements are signed	The partner's identity, and the full project brief for your project: complete feature scope, data model, technical requirements, example scenarios, glossary

Stage	What you get
	and milestone plan
At kickoff	A briefing directly from the company, sample data or assets, and access to your industry mentor

So you are not applying blind. You are applying to a problem, and the full instruction manual arrives once the paperwork allows it. Every project has a detailed brief already written and waiting.

6. How to apply

1. Read through the five projects and pick the one or two that genuinely interest you.
2. Submit an expression of interest by **3 September 2026** to **Parminder Singh (parminder.singh@digipen.edu)**, telling us which project, which role you would take, and one paragraph on why.
3. You do not need a team already formed. Individual applications are welcome, and teams are assembled afterwards.
4. Come and talk to us if you are unsure. A five-minute conversation is faster than a week of wondering.

> You do not need to know anything about electric vehicles, museums or energy systems to apply. Nobody starts knowing this. What you need is curiosity about a real problem and the willingness to learn your way into it.
>
> Pick the one that made you want to keep reading. That is usually the right answer.

Questions: **Parminder Singh — parminder.singh@digipen.edu** Deadline: **3 September 2026**